Programação 2

Aula 10 - Persistência: dados persistentes, entrada e saída de texto em arquivos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Erros de Compilação

No processo de construção do código binário

Falhas de sintaxe, por exemplo:

- **nomes errados** de variáveis, classes, métodos, etc
- **tipos** não declarados
- **; faltando**
- **ordenação das palavras reservadas**

Erros de inclusão de arquivos e bibliotecas

Erros de incompatibilidade de versões do compilador

Erros de Execução

No processo de rodar (runtime) do código binário

Em Java são chamados de **exceções**
exemplos são:

- ArrayIndexOutOfBoundsException
- ClassNotFoundException
- FileNotFoundException
- NullPointerException

Exceções

Aplicações Java pode ser multithread (rodar em mais de um processador), a mensagem de erro deixa claro qual foi a thread

Existem vários tipos de exceções. A que vimos é especificamente sobre acessos em arrays.

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
        at Main.main(Main.java:12)
```

As mensagens de exceções indicam qual foi a linha de código que lançou a exceção.

Exceções: Tratamento com **try** -> **catch**

```java
// acesso inválido
int[] c = new int[12];
c[12] = 12;

// acréscimo de um ponto para todos
for(int i = 0; i < c.length; i++)
{
    c[i] += 1;
}
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
    at Main.main(Main.java:10)
```

```java
try
{
    // acesso inválido
    int[] c = new int[12];
    c[12] = 12;

    // acréscimo de um ponto para todos
    for(int i = 0; i < c.length; i++)
    {
        c[i] += 1;
    }
}
catch(ArrayIndexOutOfBoundsException exception)
{
    System.out.println("você acessou uma posição inválida");
    System.out.println(exception);
}
```

```
você acessou uma posição inválida
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
```

Erros de Lógica/Comportamento

O código a seguir compila e roda! Mas não funciona com o esperado

```
// executando a potencia com a repetição usando while
int i = 1;
int numeroA = 2;
int potencia = 5;

while (i < potencia)
{
  numeroA = numeroA * numeroA;
  i++;
}
```

File Edit Selection View Go Run Terminal Help

Test.java - FirstProject - Visual Studio Code

RUN AND DEBUG No Configurati

VARIABLES

Local

```
args: String[0]@7
escaneador: (...)
protagonista: Personagem@9
```

WATCH

CALL STACK

Thread [Attach Listener]	RUNNING
Thread [Signal Dispatcher]	RUNNING
Thread [Finalizer]	RUNNING
Thread [Reference Handler]	RUNNING
Thread [main]	PAUSED ON BREAKPOINT
Test.main(String[]) Test.java 11:1	

BREAKPOINTS

- [ ] Uncaught Exceptions
- [ ] Caught Exceptions
- [x] Test.java src 11

Test.java 1 | App.java | Pessoa.java

src > Test.java > Test > main(String[])

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class Test
5  {
       Run | Debug
6      public static void main(String[] args)  args = String[0]@7
7      {
8          Scanner escaneador = new Scanner(System.in); escaneador = Scanner@8
9
10         Personagem protagonista = new Personagem("Julia"); protagonista = Personagem@9
11         Personagem antagonista = new Personagem("Alfa", 200);
12
13         // A partir da classe Capitulo, criar os objetos
14         Capitulo capitulo1 =
15             new Capitulo(protagonista.nome +
16             " estava correndo pela floresta e se deparou com um penhasco. O que ela fará?",
17             new String[]{"Pula", "Se esconde"},
18             protagonista, antagonista,
19             -10, 0, escaneador);
20
21         Capitulo capitulo11 =
22             new Capitulo(
23             "[Final 1] Ao pular do penhasco ela acaba se sacrificando",
24             new String[]{},
25             protagonista, antagonista,
26             -100, 0, escaneador);
27
28         Capitulo capitulo12 =
29             new Capitulo(
30             "Ao se esconder ela vê quatro lobos que estavam perseguindo ela.\n"+
31             "Claramente um deles é mais imponente, ele mentalmente apelida ele de "+ antagonista.nome+"\n"+
32             "Aparentemente os lobos se perderem, e agora?",
33             new String[]{"Sobe numa árvore", "Corre para os lobos"},
34             protagonista, antagonista,
35             0, -15, escaneador);
```

OUTPUT TERMINAL 2 DEBUG CONSOLE

Filter (e.g. text, !exclude)

Conceito de Hoje

Conceito de Hoje

Persistência: dados persistentes, entrada e saída de texto em arquivos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

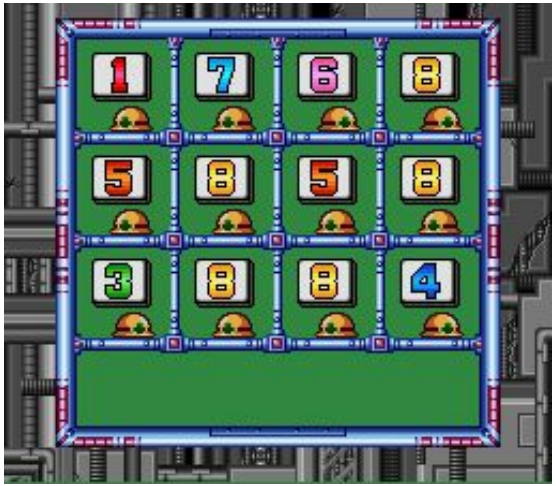
Problema

Motivação

História e Pessoas

persistência de dados

É pra começar do zero, toda vez?

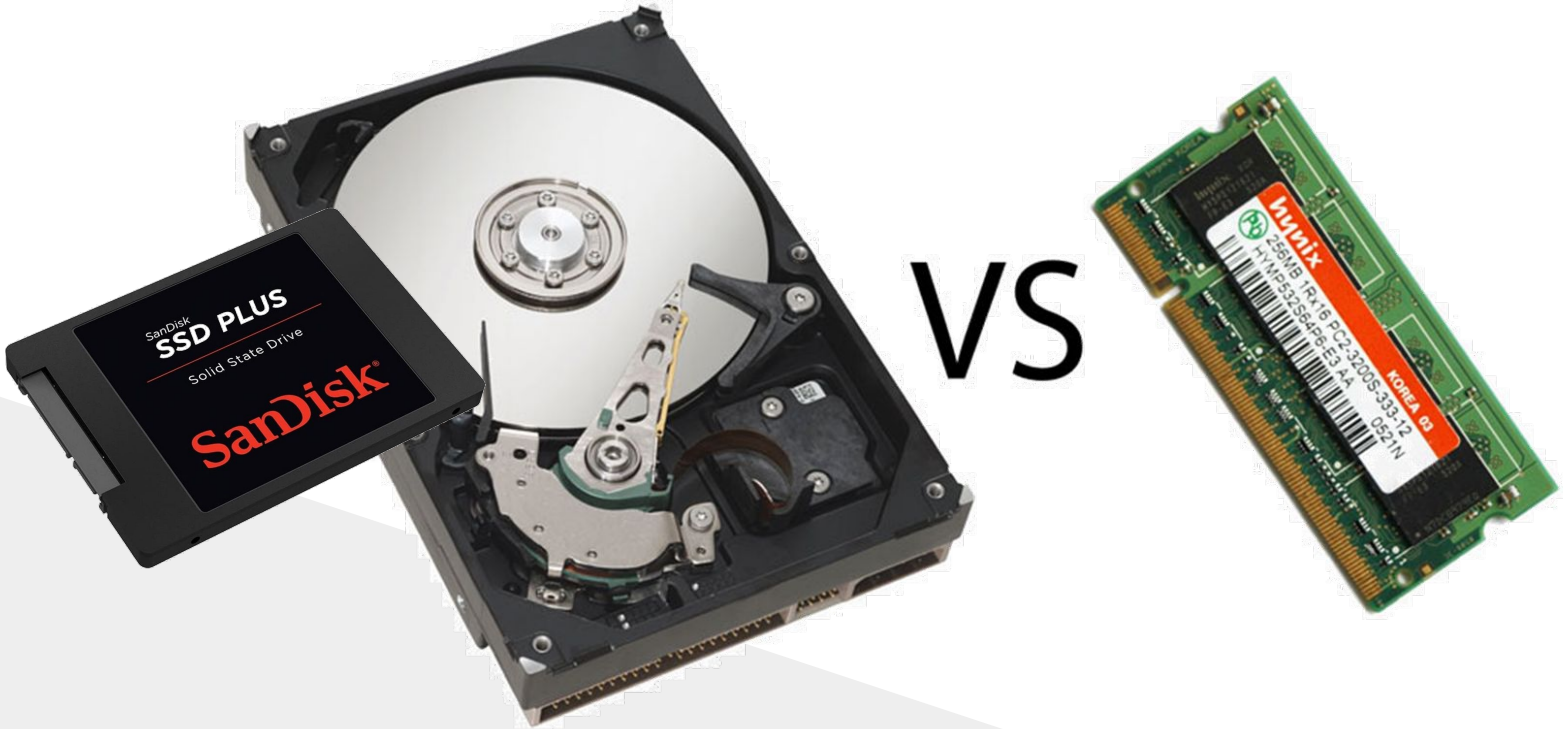


Os estados de um programa (situação atual dos dados) varia

Ao fechar o programa a memória RAM é liberada

Tem como não perder o progresso?

HD vs RAM



LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

escrita e leitura de arquivos

Leitura e Escrita de Arquivos para Persistência

Arquivos não desaparecem após fechar o programa

Todo o conteúdo da aplicação pode ser depositado em arquivos

- É uma boa prática inclusive, não incluir conteúdo no código

Neste contexto, conteúdo se refere a tudo que não define a lógica do programa

- Nomes de personagem, valores de energia, textos dos capítulos, etc
- Imagens
- Vídeos
- Modelos 3D
- Etc

Criando Arquivos em Java

Classe File

```
try {
  File myObj = new File("filename.txt");
  if (myObj.createNewFile()) {
    System.out.println("File created: " + myObj.getName());
  } else {
    System.out.println("File already exists.");
  }
} catch (IOException e) {
  System.out.println("An error occurred.");
  e.printStackTrace();
}
```

Java Create and Write To Files (w3schools.com)

Escrevendo em Arquivos Java com FileWriter

```java
try {
  FileWriter myWriter = new FileWriter("filename.txt");
  myWriter.write("Files in Java might be tricky, but it is fun enough!");
  myWriter.close();
  System.out.println("Successfully wrote to the file.");
} catch (IOException e) {
  System.out.println("An error occurred.");
  e.printStackTrace();
}
```

[Java Create and Write To Files (w3schools.com)]

Lendo Arquivos em Java com Scanner

File + Scanner

```java
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class Test
{
    Run | Debug
    public static void main(String[] args)
    {
        File arquivo = new File("rsc/Arquivo.txt");

        try {
            Scanner escaneador = new Scanner(arquivo, "UTF-8");

            int i = 0;
            while (escaneador.hasNextLine()) {
                i++;
                String leitura = escaneador.nextLine();
                System.out.println("Linha "+ i + ": " + leitura);
            }
            escaneador.close();
        }
        catch (FileNotFoundException e) {
            e.printStackTrace();
        }
```

Arquivo

xspace\Java\Projects\First Project\FirstProject\rsc\Arquivo.txt - Sublime Text (UNREGISTERED)

```
1  Esta é a primeira linha
2  Esta é a segunda...
3  .
4  .
5  Esta é a quinta e última linha
```

Console

d:\Workspace\Java\Projects\First Project\FirstProject - VS Code Console

```
Linha 1: Esta é a primeira linha
Linha 2: Esta é a segunda...
Linha 3: .
Linha 4: .
Linha 5: Esta é a quinta e última linha
Press any key to continue . . .
```

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Escrevendo em Arquivos Java com FileReader

FileReader

```java
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;

public class Test
{
    Run | Debug
    public static void main(String[] args)
    {
        File arquivo = new File("rsc/Arquivo.txt");

        try {
            Scanner escaneador = new Scanner(arquivo, "UTF-8");

            int i = 0;
            while (escaneador.hasNextLine()) {
                i++;
                String leitura = escaneador.nextLine();
                System.out.println("Linha "+ i + ": " + leitura);
            }
            escaneador.close();
        }
        catch (FileNotFoundException e) {
            e.printStackTrace();
        }
```

Arquivo

xspace\Java\Projects\First Project\FirstProject\rsc\Arquivo.txt - Sublime Text (UNREGISTERED)

```
1  Esta é a primeira linha
2  Esta é a segunda...
3  .
4  .
5  Esta é a quinta e última linha
```

Console

d:\Workspace\Java\Projects\First Project\FirstProject - VS Code Console

```
Linha 1: Esta é a primeira linha
Linha 2: Esta é a segunda...
Linha 3: .
Linha 4: .
Linha 5: Esta é a quinta e última linha
Press any key to continue . . .
```

dicionários

Dicionários (HashMap)

Arrays são acessados pelo índice, mas e se quiséssemos acessar cada item pelo nome do personagem. Ou seja, usar o nome como a "**chave**" de acesso.

```
Map<String, Personagem> personagens = new HashMap<String, Personagem>();
personagens.put("Julia", new Personagem("Julia"));
personagens.put("Julia", new Personagem("Julia"));
personagens.put("André", new Personagem("André"));
personagens.put("Alfa", new Personagem("Alfa"));

personagens.get("Julia");

for (Map.Entry<String,Personagem> personagem : personagens.entrySet()) {
    System.out.println("Chave: " + personagem.getKey());
}
```

Extras

JSON e Serialização

Formato definido

Utilizado mundialmente

Utilizado para serialização

- Escrita de objetos em arquivos
- Leitura de arquivos para objetos

testJSON 〉 testJSON 〉 data.json 〉 No Se

```
1  {
2      "users": [
3              {
4              "name": "John",
5              "age": 25
6              },
7              {
8              "name": "Mark",
9              "age": 29
10             },
11             {
12             "name": "Sarah",
13             "age": 22
14             }
15             ],
16     "dataTitle": "JSON Tutorial!",
17     "swiftVersion": 2.1
18 }
```

[GitHub - google/gson: A Java serialization/deserialization library to convert Java Objects into JSON and back]

Exercício

Mini-Projeto 6

Ter um **arquivo que armazena todos os dados dos personagens** da história.

Usar o arquivo para carregar os personagens no código, armazenar em um **dicionário de personagens** (HashMap).

Ter um **arquivo que armazena todos os dados dos capítulos da história**, incluindo as escolhas, os nomes dos personagens e a variação de energia deles. Você precisará atribuir um nome aos capítulos, para nas escolhas deixar especificado o capítulo seguinte.

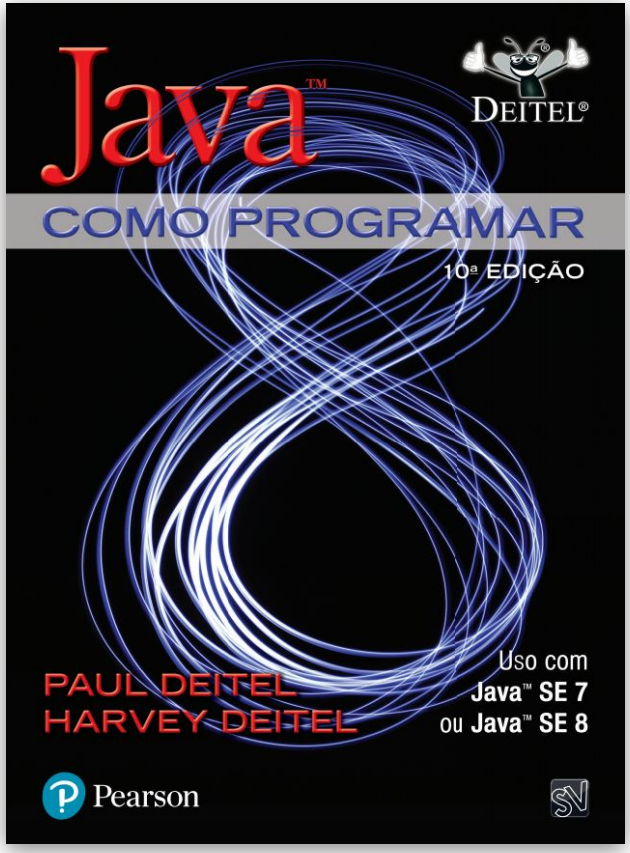
Usar o arquivo para carregar os capítulos no código, armazenar em um **dicionário de capítulos**.
O carregamento dos capítulos deve usar os personagens já carregados, acessando eles no dicionário pelo nome.

Desta forma, **todo o conteúdo do livro interativo deve estar armazenado em arquivos** de recursos (ex.: .txt), nenhum texto dos capítulos, texto das escolhas, nomes e energias dos personagens deverá estar explicitamente escrito no código (.java).

[Extra + 1pt na 2VA] Salvar o progresso da história ao mostrar cada capítulo, de forma que ao desligar a aplicação e ligar novamente o leitor/jogador continuará de onde parou.

Referências

Java - Como Programar (10a Edição)

Ferramenta:

Eclipse: eclipse.org/downloads